

Zero-Knowledge Proof

Interactive Proof

What is a Proof?

- **Classical proof (just the intuition, not the definition):**

The **way** to verify a statement is correct.

- Given y , there exists some x such that $x^2 \equiv y \pmod{n}$.



The Requirement of a Proof

- **Completeness:** If the statement is true, prover will convince honest Verifier.
- **Soundness:** If the statement is false, honest Verifier won't accept any purported proof.

But if I want to prove some abstract thing...

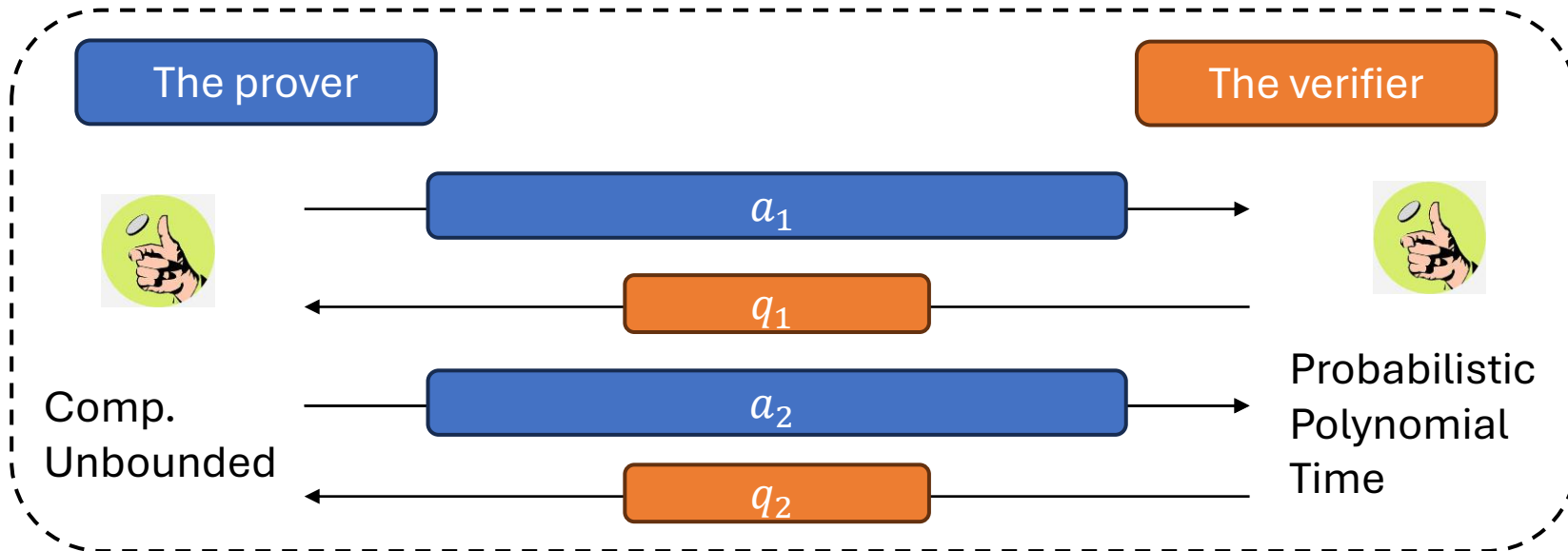
that is even hard to describe, such that:

- I can tell the difference between Coca-Cola and Pepsi.
- My twin sister and I have a telepathic connection.

Two New Ingredients

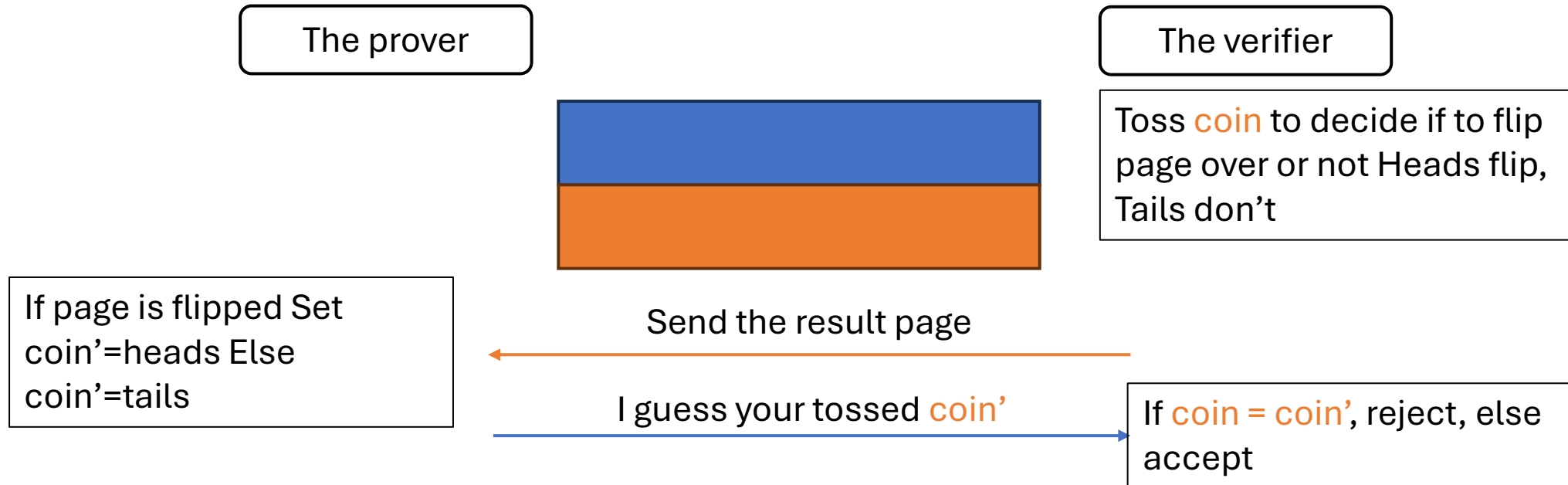
Interaction: rather than passively “reading” proof, verifier engages in a non-trivial interaction with the prover.

Randomness: verifier is randomized (tosses coins as a primitive operation), and can err in accept/reject with small probability.



An Example: Proof of Different Colors

- How to convince that two colors are different to a **blind verifier**?

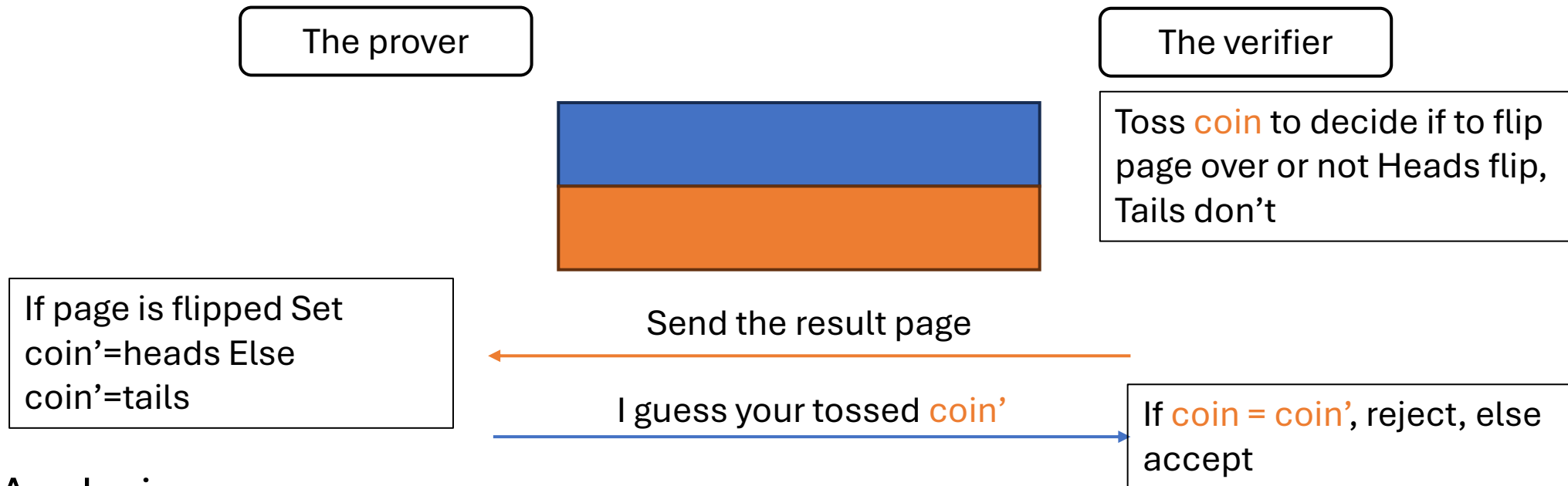


Interactive Proof

- Goal: To prove x is in a language L .
- All powerful prover, probabilistic-polynomial-time verifier.
- **Completeness:** If x in L , prover will convince honest Verifier, with probability 1.
- **Soundness:** If x not in L , honest Verifier won't accept any purported proof, except for probability $negl(|x|)$.

An Example: Proof of Different Colors

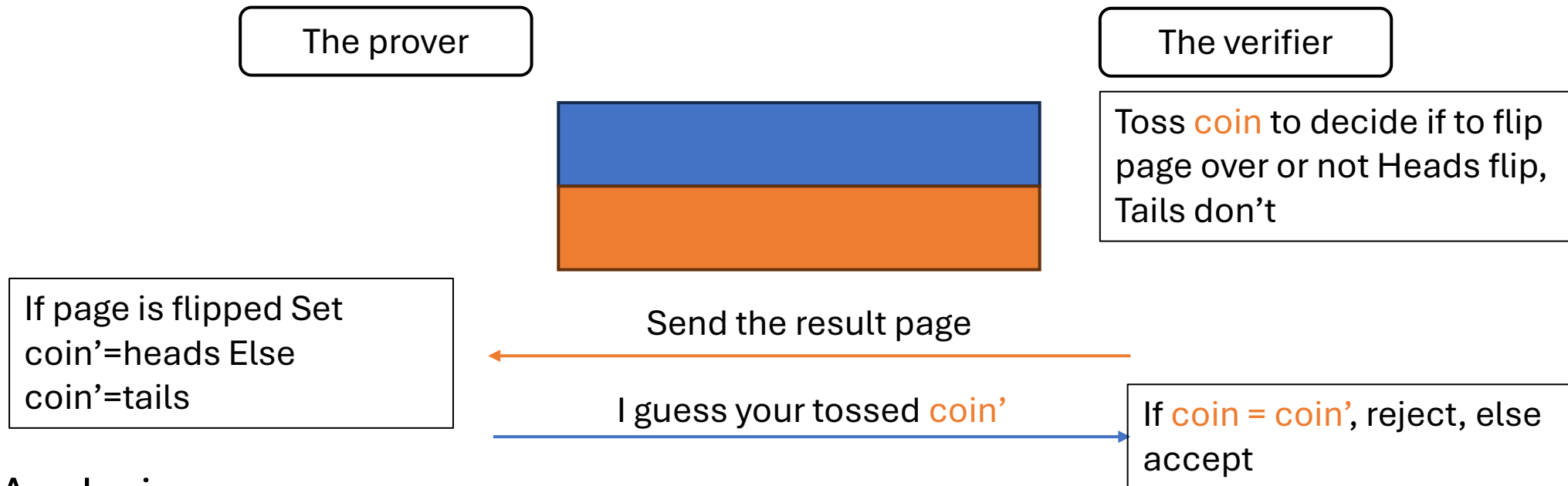
- How to convince that two colors are different to a **blind verifier**?



- Analysis:
 - If there are 2 colors, the verifier accepts with prob. 1.
 - If there is 1 color, the verifier only accepts with prob. $\frac{1}{2}$.

An Example: Proof of Different Colors

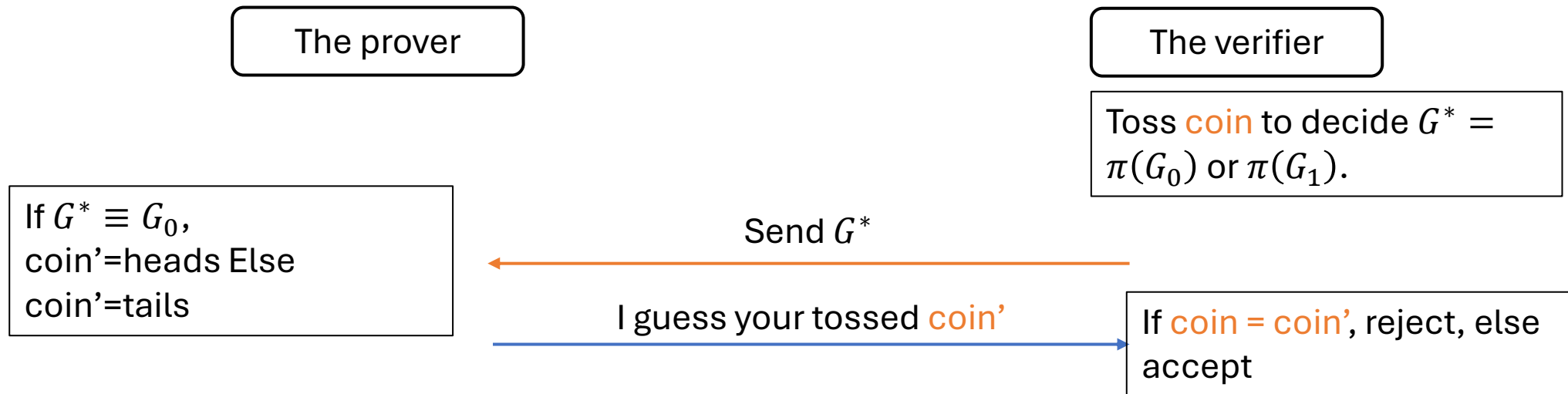
- How to convince that two colors are different to a **blind verifier**?



- Analysis:**
 - If there are 2 colors, the verifier accepts with prob. 1.
 - If there is 1 color, the verifier only accepts with prob. $\left(\frac{1}{2}\right)^k$, after repeating k times.

An Example: Graph Non-Isomorphism

- How to convince V that two graphs G_1 and G_2 are not isomorphism?



- Analysis:
 - If they are not isomorphic, the verifier accepts with prob. 1.
 - If they are isomorphic, the verifier only accepts with prob. $\left(\frac{1}{2}\right)^k$, after repeating k times.

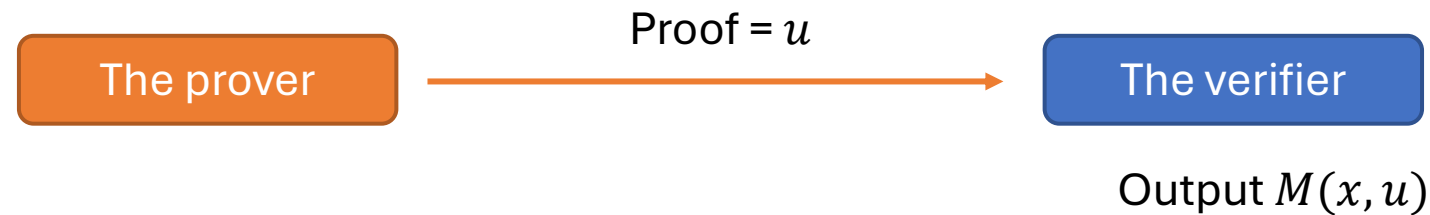
Interactive Proof of NP Language

Definition 2.1 (*The class NP*)

A language $L \subseteq \{0, 1\}^*$ is in **NP** if there exists a polynomial $p : \mathbb{N} \rightarrow \mathbb{N}$ and a polynomial-time TM M (called the *verifier* for L) such that for every $x \in \{0, 1\}^*$,

$$x \in L \Leftrightarrow \exists u \in \{0, 1\}^{p(|x|)} \text{ s.t. } M(x, u) = 1$$

If $x \in L$ and $u \in \{0, 1\}^{p(|x|)}$ satisfy $M(x, u) = 1$, then we call u a *certificate* for x (with respect to the language L and machine M).

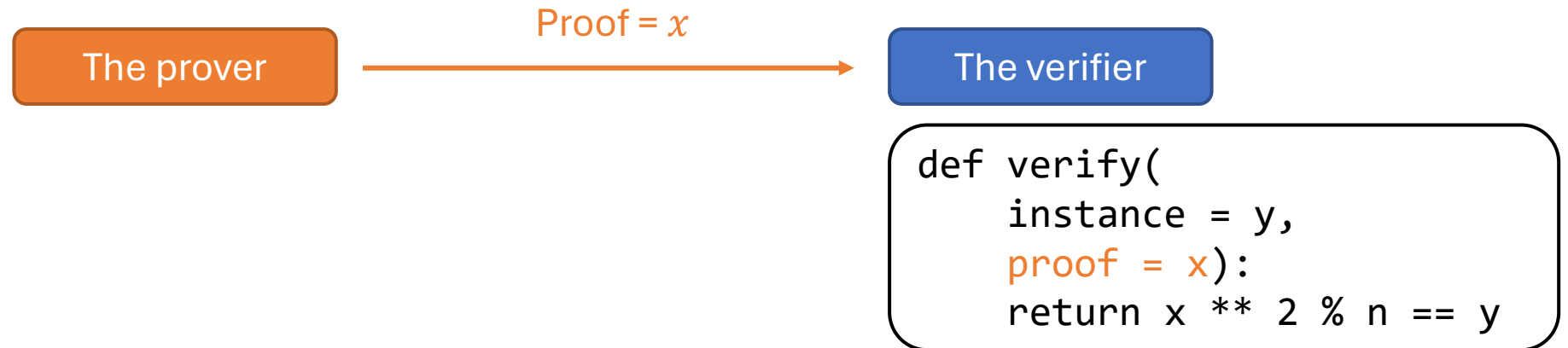


Zero-Knowledge Proof

Protect the Knowledge from Verifier

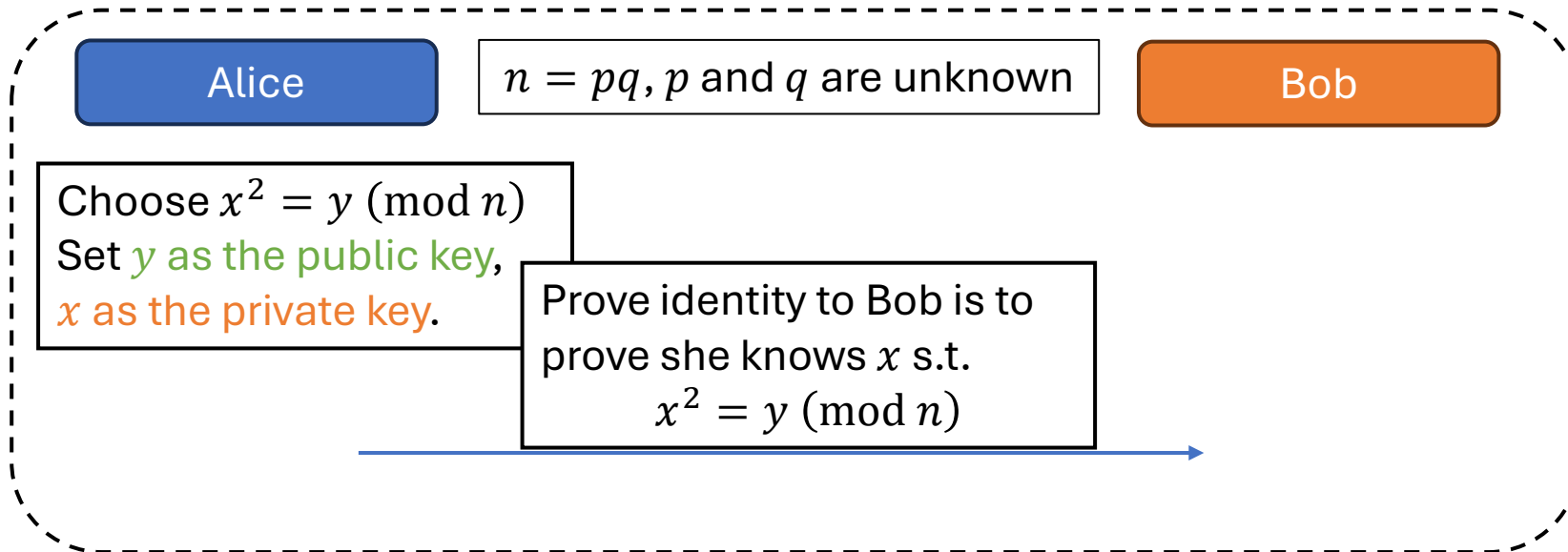
- Example 1:

- **Definition:** y is a quadratic residue modulo n if $\exists w$ s.t. $x^2 \equiv y \pmod{n}$.
- **Statement:** y is a quadratic residue modulo n .



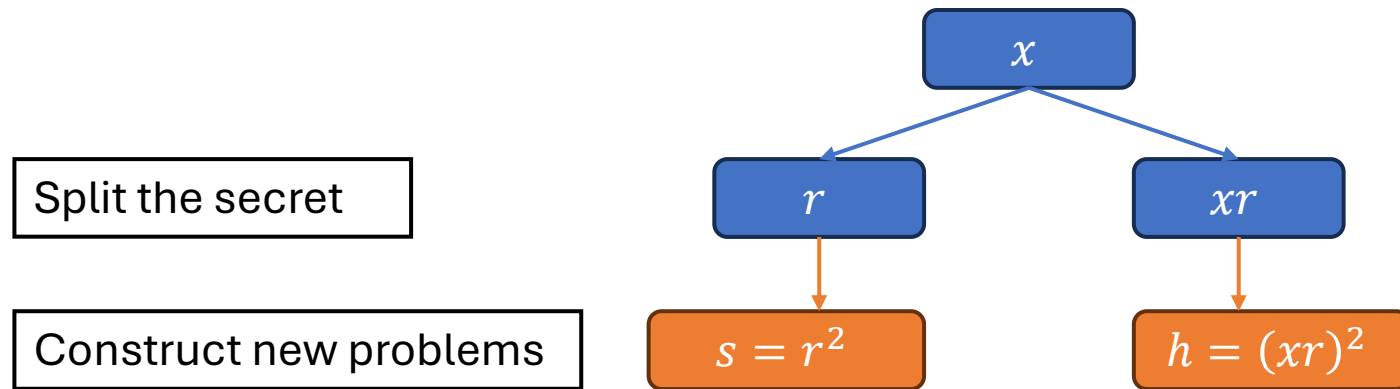
Protect the Knowledge from Verifier

- However, what if we don't want to disclose the proof?
 - In the previous example, the **square root x** , or the **mapping π** .
- Example: **identification protocol**:
 - quadratic residuosity problem: it's hard to find x s.t. $x^2 = y \pmod{n}$, $n = pq$.



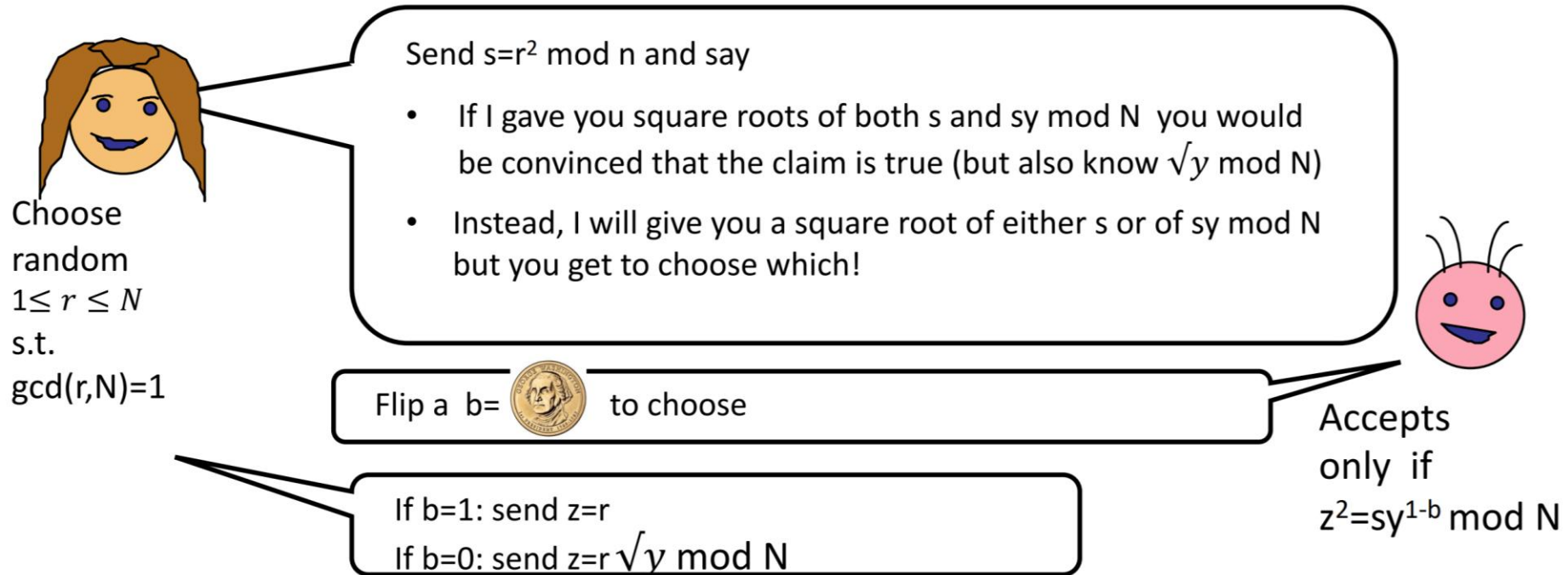
Protocol without Leaking x : Intuition

- To prove $x^2 \equiv y \pmod{n}$.
- Split the secret s into two parts
 - Randomly select some r .
 - “Share” the secret of x be r and xr .



- If we can answer both $\sqrt{s} = r$ and $\sqrt{h} = xr$, we can recover $x = r^{-1} * (xr)$.
- However, we only allow the verifier to **query one of them!**

Interactive Proof for QR



Analysis:

If there are x s.t. $x^2 = y$, the verifier accepts with prob. 1.

Otherwise, the verifier only accepts with prob. $\left(\frac{1}{2}\right)^k$, after repeating k times.

Intuition of Zero-Knowledge Property

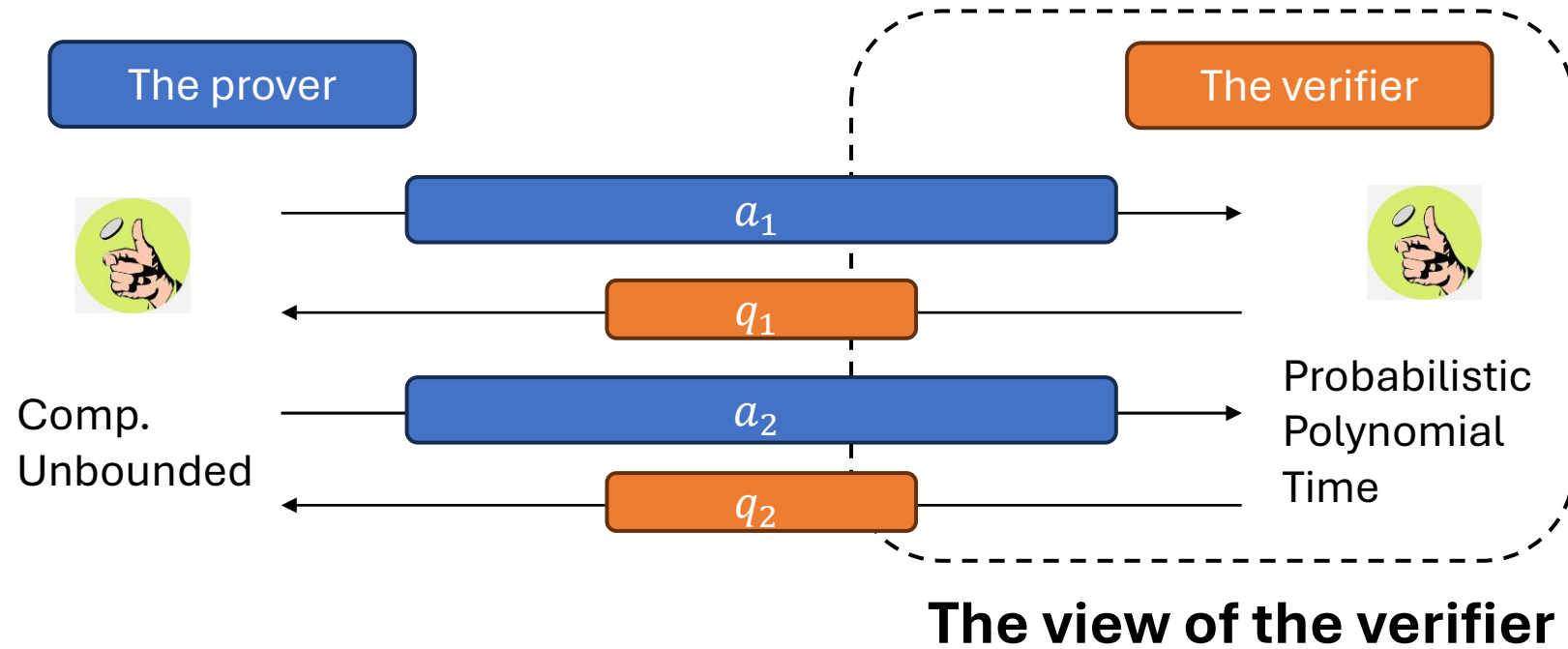
What the verifier can compute **after the interaction**

=

What the verifier could have computed **before interaction**

How do we capture this mathematically?

The Verifier's View



After the interactive proof, the verifier knows:

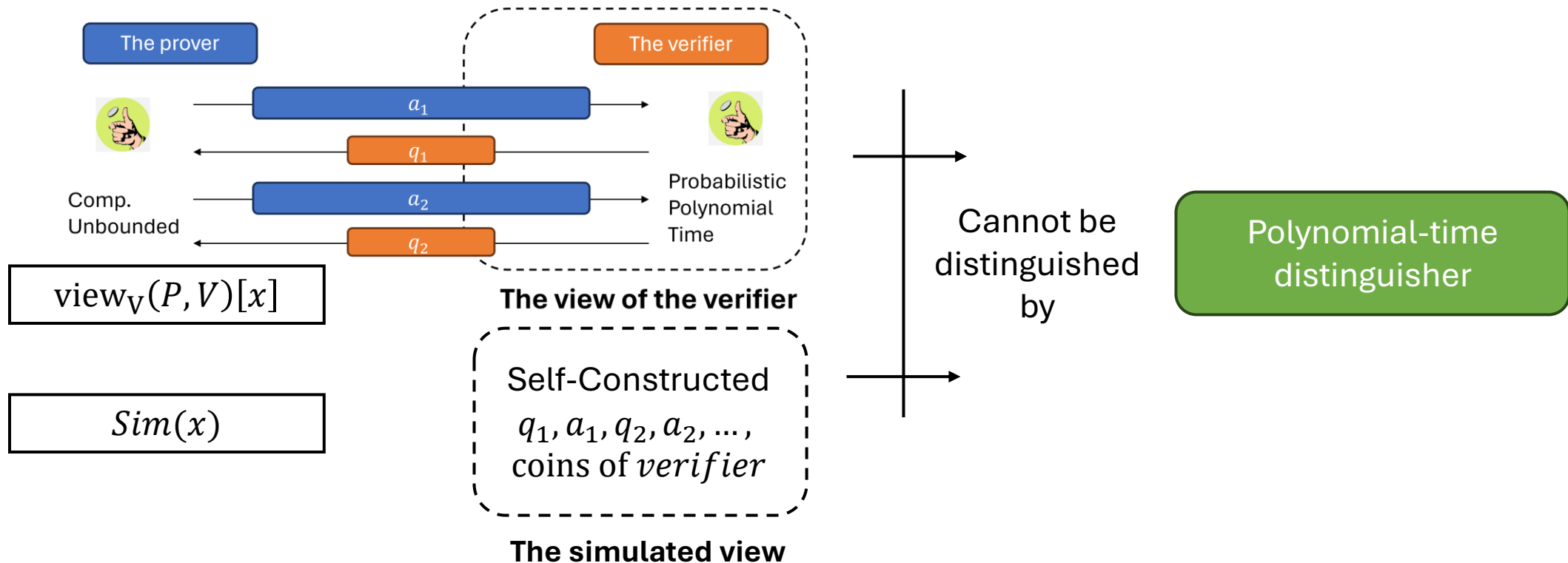
- The statement is true.
- A view of interaction, defined as:

$$\text{view}_V(P, V)[x] = (q_1, a_1, q_2, a_2, \dots, \text{coins of } V)$$

which is a random variable controlled by P and V's coins.

The Simulation Paradigm

- V's view gives him nothing new, if he could have simulated it its own s.t **simulated view** and **real-view** are **computationally-Indistinguishable**.



Zero-Knowledge Proof: Definition

- **Completeness:** If x in L , prover will convince honest Verifier, with probability 1.
- **Soundness:** If x not in L , honest Verifier won't accept any purported proof, except for probability $negl(|x|)$.
- **Honest-Verifier Zero-knowledgeness:** there exists a **PPT algorithm** Sim (a simulator) such that for every $x \in L$, the following two probability distributions are **poly-time indistinguishable**:

$$\text{view}_V(P, V)[x, 1^\lambda]$$

$$Sim(x, 1^\lambda)$$

Zero-Knowledge Proof: Definition

- **Completeness:** If x in L , prover will convince honest Verifier, with probability 1.
- **Soundness:** If x not in L , honest Verifier won't accept any purported proof, except for probability $negl(|x|)$.
- **Zero-knowledgeness:** if **for every PPT V^*** there exists a **PPT algorithm** Sim (a simulator) such that for every $x \in L$, the following two probability distributions are **poly-time indistinguishable**:

$$\text{view}_V(P, V^*)[x, 1^\lambda]$$

$$\text{Sim}(x, 1^\lambda)$$

Flavors of Zero-Knowledge

$$\boxed{\text{view}_V(P, V)[x, 1^\lambda]} \approx \boxed{\text{Sim}(x, 1^\lambda)}$$

- Computationally indistinguishable distributions = CZK
- Perfectly identical distributions = PZK
- Statistically close distributions = SZK

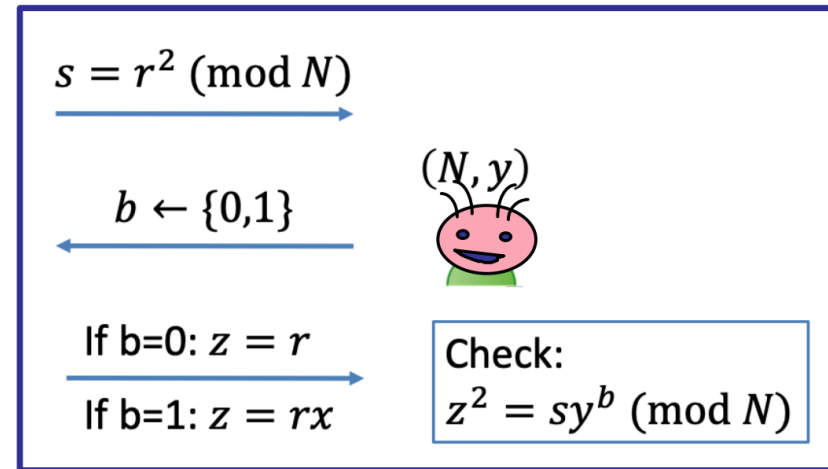
The **total variation distance** of two distributions X and Y over a finite domain D (often referred to as *statistical difference* or *statistical distance* in cryptography) is defined as

$$\Delta(X, Y) = \frac{1}{2} \sum_{\alpha \in D} |\Pr[X = \alpha] - \Pr[Y = \alpha]|$$

We say X and Y are **statistically close** if the distance is negligible.

A Simulation for QR Protocol

$view_V(P, V):$
Transcript = (s, b, z) ,
Coins = b

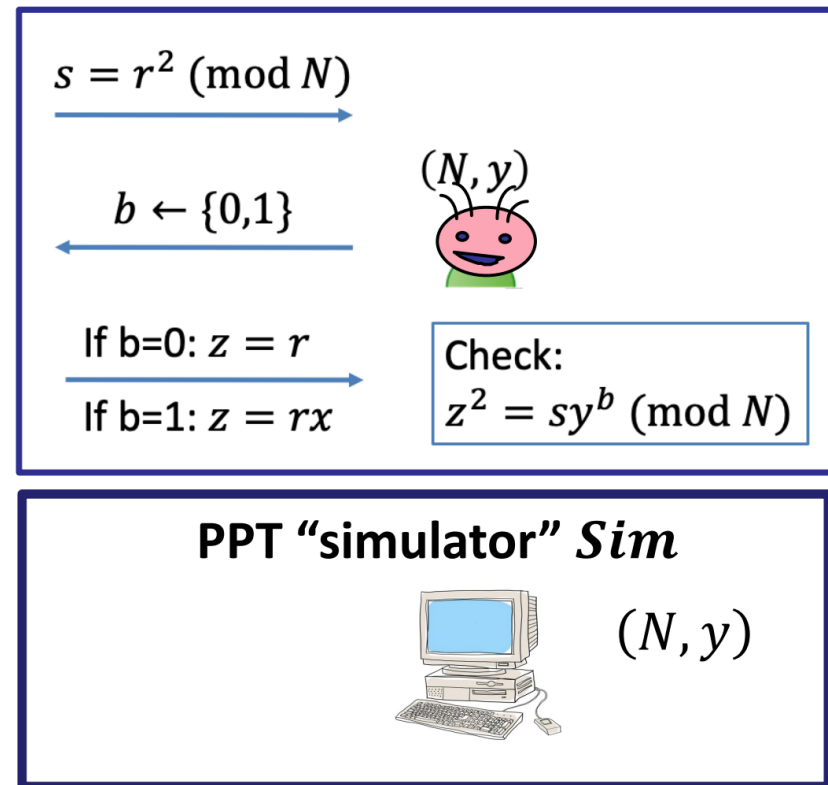


A Simulation for QR Protocol



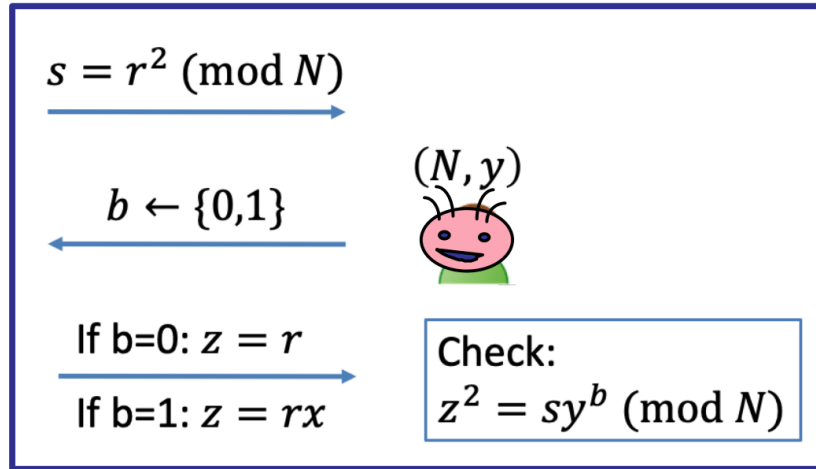
$view_V(P, V):$
 (s, b, z)

$sim :$
 (s, b, z)



A Simulation for QR Protocol

Claim: The QR protocol is perfect zero knowledge.



Simulator S works as follows:

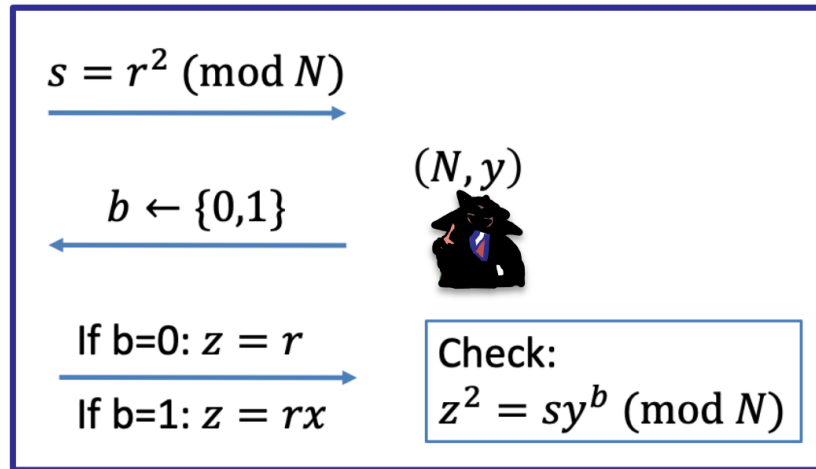
1. First pick a random bit b .
2. pick a random $z \in Z_N^*$.
3. compute $s = z^2 / y^b$.
4. output (s, b, z) .

$view_V(P, V):$
 (s, b, z)

claim: The simulated transcript is identically distributed as the real transcript

A Simulation for QR Protocol

Claim: The QR protocol is perfect zero knowledge.



$view_V(P, V):$
 (s, b, z)

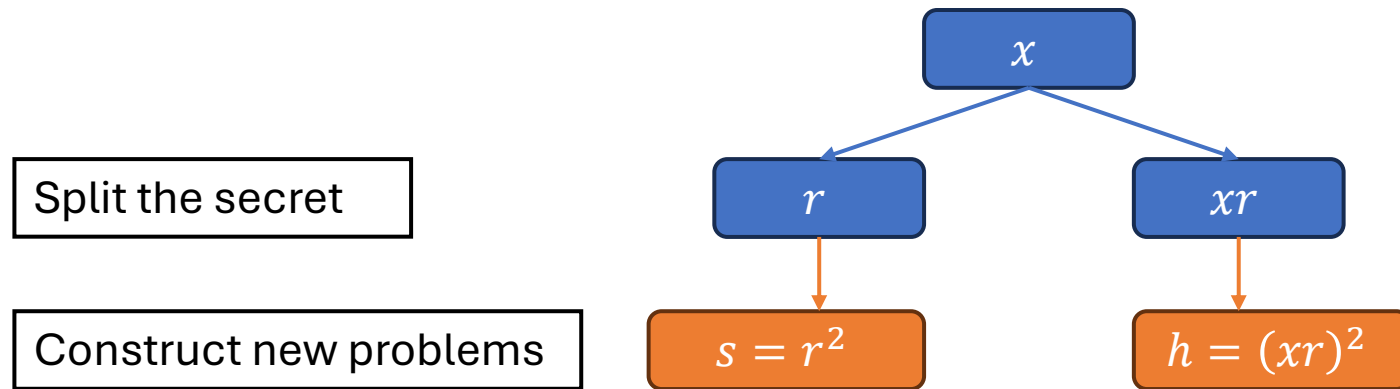
Simulator S works as follows:

1. First pick a random bit b .
2. pick a random $z \in Z_N^*$.
3. compute $s = z^2 / y^b$.
4. If $V^*((N, y), s) = b$ output (s, b, z)
if not goto 1 and repeat

Claim: Expected number of repetitions is two

This Construction Prove the existence of x .

- To prove $x^2 \equiv y \pmod{n}$.
- Split the secret s into two parts
 - Randomly select some r .
 - “Share” the secret of x be r and xr .



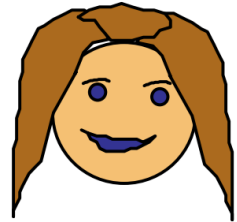
- If we can answer both $\sqrt{s} = r$ and $\sqrt{h} = xr$, we can recover $x = r^{-1} * (xr)$.
- However, we only allow the verifier to **query one of them!**

Proof of Knowledge

- Consider $L_R = \{x : \exists w, s.t. R(x, w) = \textit{accept}\}$ for poly-time relation R .
- **Definition:** (P, V) is a proof of knowledge (POK) for L_R if : $\exists$ PPT (knowledge) extractor algorithm E s.t. $\forall x \in L$, in expected poly-time $E^P(x)$ outputs w s.t. $R(x, w) = \textit{accept}$.

The Rewinding Method of QR Protocol

Input: (y, N)



Extractor
Algorithm

$s = r^2 \bmod N$



head



r



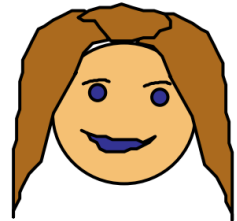
Extractor:

On input (y, N) ,

1. Run prover & receive s
2. Set verifier message to **head**; Store r

The Rewinding Method of QR Protocol

Input: (y, N)



Extractor
Algorithm

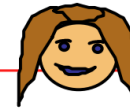
$s = r^2 \bmod N$



tail



$rx \bmod N$

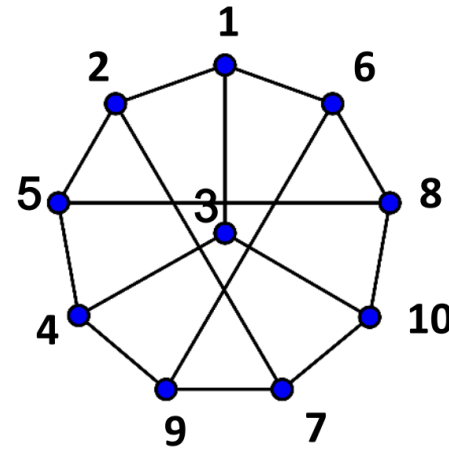
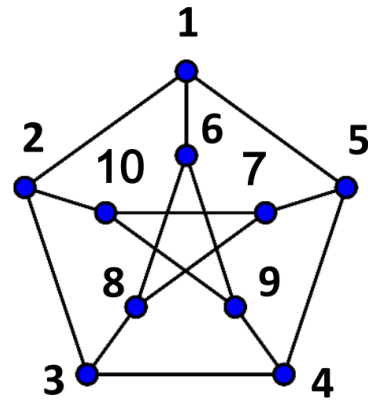


Extractor:

On input (y, N)

1. Run prover & receive s
2. Set verifier message to **head**; receive and store r
3. **Rewind** and 2nd time set verifier message to **tail** receive rx
4. Output $rx/r = x \bmod N$

ZK Proof for Graph Isomorphism



Recall:

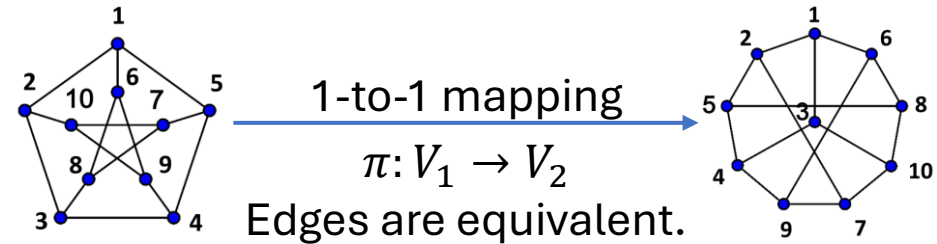
G_0 is isomorphic to G_1

If $\exists$ isomorphism $\pi: [N] \rightarrow [N]$, $\forall i, j: (\pi(i), \pi(j)) \in E_1$ iff $(i, j) \in E_0$.

Trivial Proof of Graph Isomorphism

- **Definition:** Graph isomorphism between $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$.

- **Statement:** $G_1 \equiv G_2$.



The prover

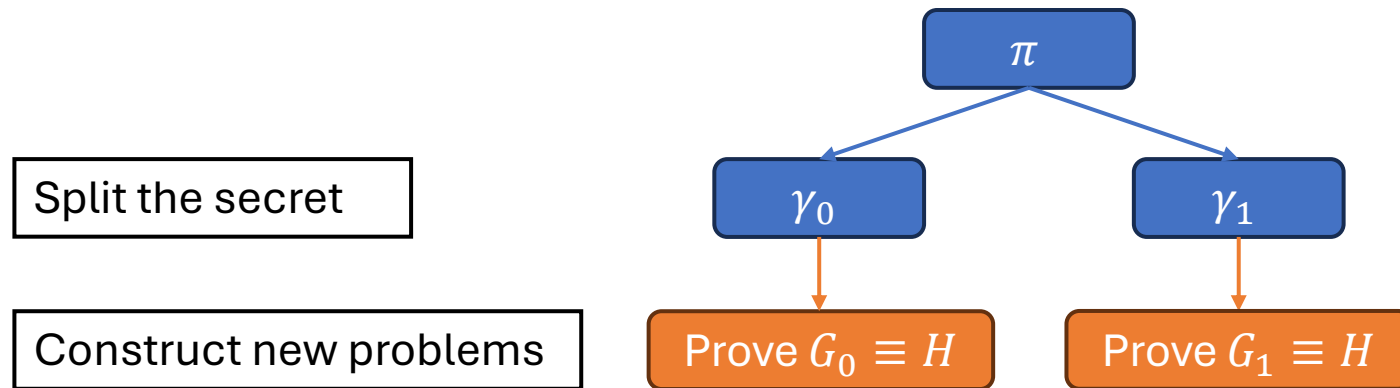
Proof = π

The verifier

```
def verify(  
    instance = (G1, G2),  
    proof = pi):  
    1. pi is 1-to-1.  
    2. (v1, v2) in E1 ==  
        (pi(v1), pi(v2)) in E2
```

This Construction Prove the existence of π .

- To prove $G_2 = \pi(G_1)$.
- Split the secret π into two parts
 - Randomly select permutation $\gamma_0, H = \gamma_0(G_0) = \gamma_1(G_1)$.
 - “Share” the secret of π be γ_0 and $\gamma_1, \pi = \gamma_0\gamma_1^{-1}$.



- If we can answer both γ_0 and γ_1 then $\pi = \gamma_0\gamma_1^{-1}$.
- However, we only allow the verifier to **query one of them!**

ZK Proof for Graph Isomorphism



Proof:

$H = \gamma_0(G_0)$,

$H = \gamma_1(G_1)$,

Thus

$G_1 = \gamma_1^{-1}(\gamma_0(G_0))$

Set $\sigma = \gamma_1^{-1} \gamma_0$

I will produce a random graph H for which

1: I can give an isomorphism γ_0 from G_0 to H

OR

2: I can give an isomorphism γ_1 from G_1 to H

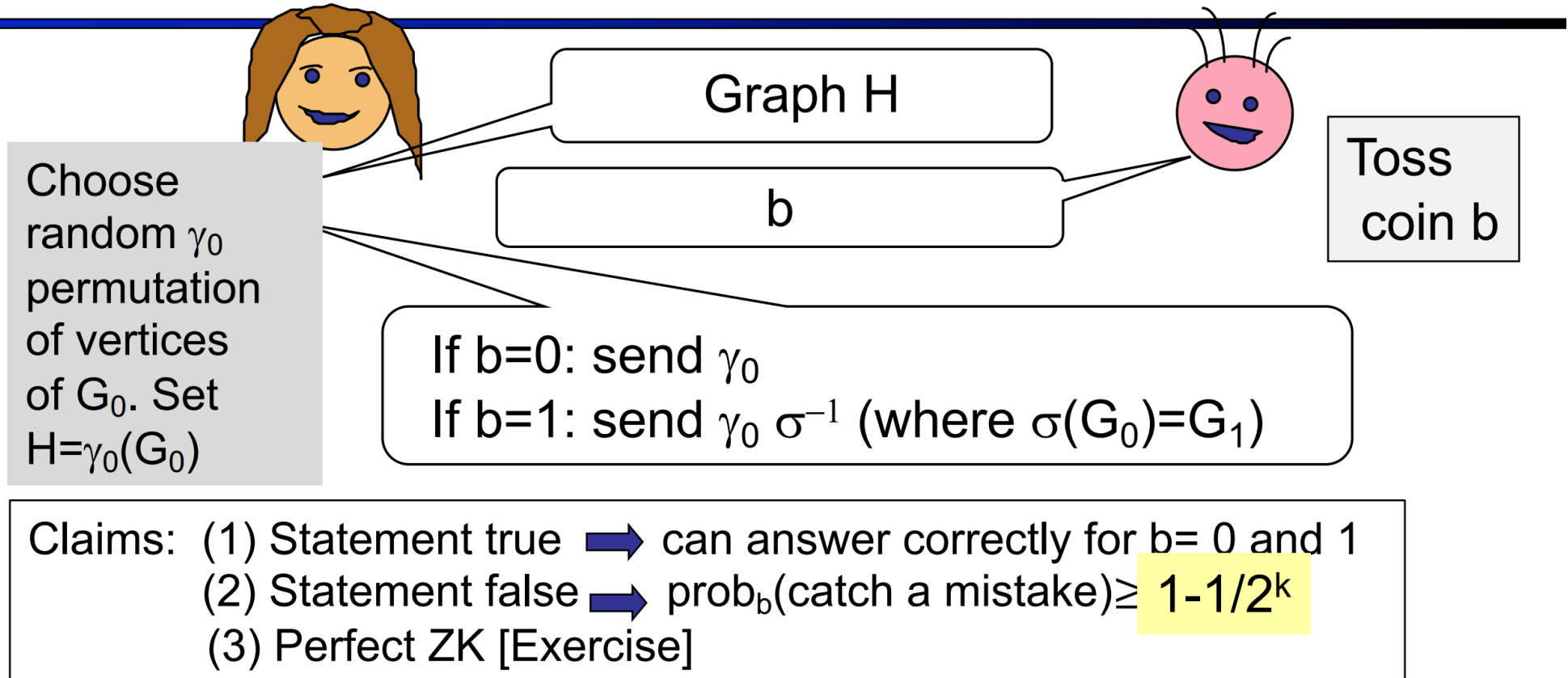
Thus, $\exists$ isomorphism σ from G_0 to G_1

Verifier, please randomly choose if I should demonstrate my ability to do **#1** or **#2**.

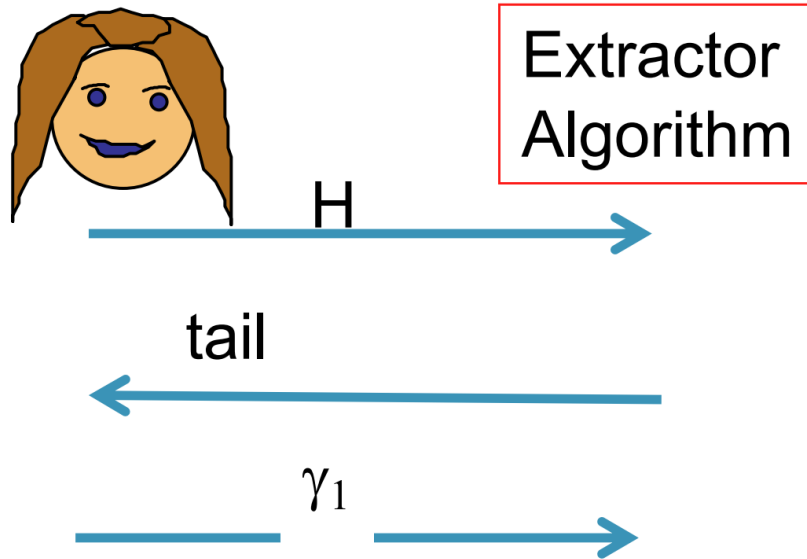


POINT IS: If I can do both,
there exists an isomorphism from G_0 to G_1

ZK Proof for Graph Isomorphism



ZK Proof for Graph Isomorphism



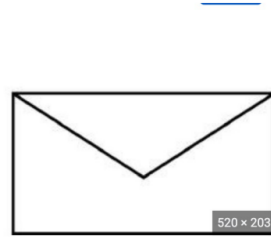
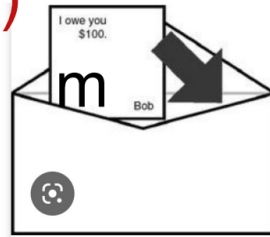
Extractor :

- 1) On input H
set **coin**=head
Store γ_0
- 2) **Rewind** and 2nd time
set **coin**=tail
Store γ_1
- 3) Output $\gamma_1^{-1}(\gamma_0)$

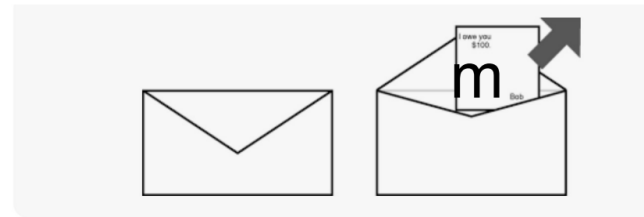
Do all NP Languages have ZKPs?

- Yes! Computational ZK.
- Suppose one-way function exists.
- Firstly, construct a commitment scheme:

Commit(m)



Decommit



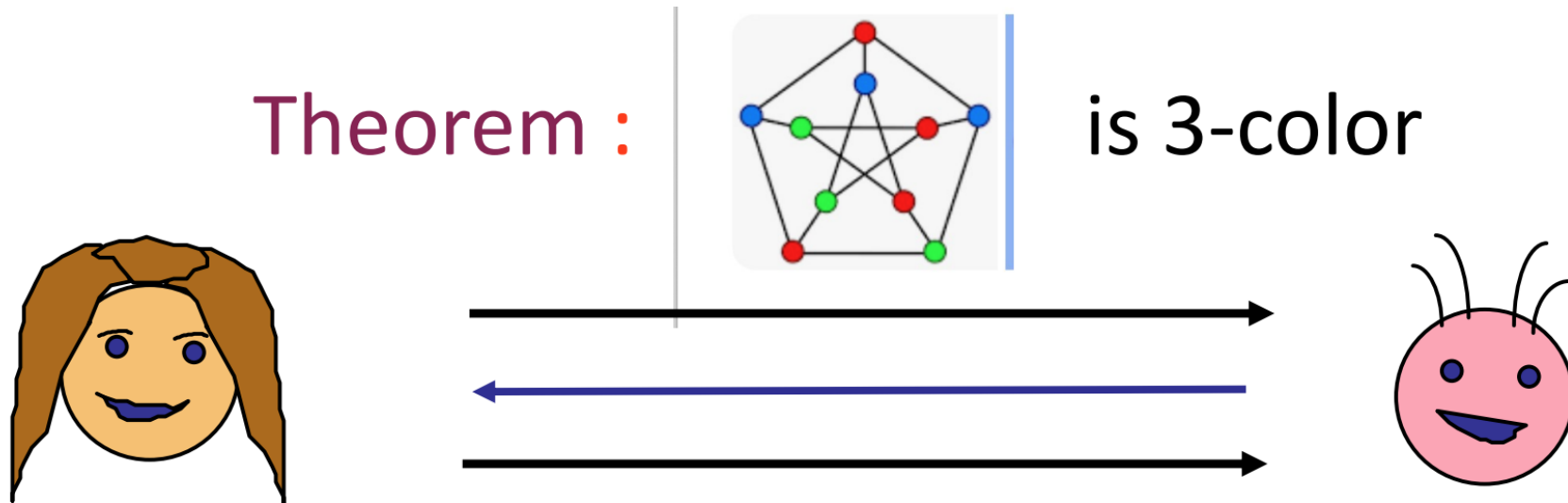
hiding

binding

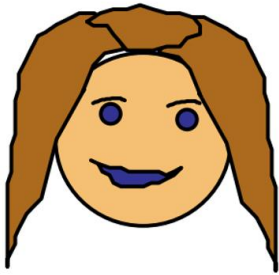
- **Hiding** (intuition): The commitment doesn't leak the message m .
- **Binding** (intuition): After commit and decommit stage, it's impossible to accept a different m' different from the original m .

G3-COLOR has a ZK interactive Proof

Graph K-coloring Problem: A K-coloring problem for undirected graphs is an assignment of colors to the nodes of the graph such that **no two adjacent vertices have the same color**, and **at most K** colors are used to complete color the graph.



G3-COLOR has a ZK interactive Proof



A coloring $\pi: V \rightarrow \{0, 1, 2\}$

Pick a random permutation σ
of colors, color the graph
 $\phi(v) = \sigma(\pi(v))$,

Commit $\phi(v)$ for all v

A random edge (u, v)

Open $\phi(u)$ and $\phi(v)$.



Reject if $\phi(u) = \phi(v)$.

Completeness and Soundness

- **Completeness:** if G is 3-colorable, then the honest prover uses a proper 3-coloring & the verifier always accepts.
- **Soundness:** If G is not 3-colorable, then for all P^* ,
$$\text{Prob}[\text{Verifier accepts}] < (1 - 1/|E|)^k < 1/e^{|E|}$$

for $k = |E|^2$.
- **Zero Knowledge:** Easy to see informally, Messy to prove formally

Honest Verifier Computational ZK

Simulator S in input $G=(V,E)$: choose at random in advance a challenge (a,b) of the honest verifier V .

- Choose random edge (a,b) in G
- Choose colors ϕ_a, ϕ_b in $\{0,1,2\}$ s.t $\phi_a \neq \phi_b$ at random and for all other $v \neq a,b$ set $\phi_v = 2$. Output simulated-view =
(commit-transcript to $\phi(v)$ for all v , edge $= (a, b)$,
decommit-transcript to colors ϕ_a, ϕ_b)

Computational ZK: Simulation for any Verifier

Simulator S on input G and verifier V^* : For $i = 1$ to $|E|^2$:

- Choose random edge (a, b) and generate commitments com to colors as in honest verifier simulation.
- Run V^* on com to obtain challenge (a^*, b^*) ;
if $(a^*, b^*) = (a, b)$, then output simulation as honest verifier case,
If all iterations fail, then output $\perp$.

Claim: If Commitment scheme is Hiding & Binding, then

$\forall G, \pi$ (a true coloring) : $\text{prob}[\perp \text{ output}] = \text{neg}(|E|)$ and if $\perp$ is not output, then
simulated-view $\approx_c$ real-view